

PilotFish End of Line Conversion Processor – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

This step **converts line endings** in the transaction data stream to a specified sequence (e.g., `\r\n` for Windows, `\n` for Unix). Reads line-by-line and appends the configured end-of-line sequence.

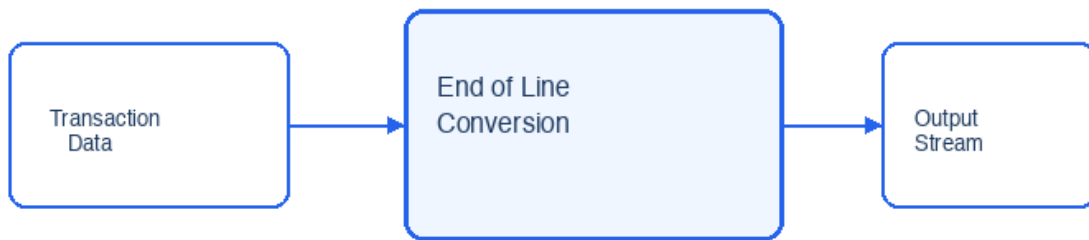


Figure 1: At a glance (plain English)

- **Reads** the data stream line by line using `BufferedReader.readLine()`.
- **Appends** the configured end-of-line sequence after each line.
- **Supports** escape sequences: `\\r`, `\\n`, `\\t`.
- **Replaces** the transaction stream with the re-formatted content.

Purpose

Deep dive into **End of Line Conversion** (EOLProcessor) in `eip.war.hs.26R1.11`.

Provenance	Value
WAR	<code>eip.war.hs.26R1.11</code>
Module JAR	<code>xcs-modules-file-1.0.1.jar</code>
Implementation class	<code>com.pilotfish.eip.modules.file.EOLProcessor</code>
Type string	End of Line Conversion
Categories	FORMAT_CONVERSION

1. Conceptual model

Data stream with mixed line endings

```
|
v
End of Line Conversion Processor
|
+-- readLine() strips original EOL
+-- append configured sequence
|
v
Uniformly-terminated lines -> setDataStream
```

2. High-level behavior (processData)

1. Read configured **Endline Sequence**; replace `\\r -> \\r`, `\\n -> \\n`, `\\t -> \\t`.
2. Create EIP cache and writer.
3. Read data stream line-by-line via `BufferedReader`.
4. Write each line + endline sequence to cache writer.
5. Close writer; set data stream to cache input stream.
6. Return `data`.

3. Configuration reference

Endline Sequence

Property	Value
UI label	Endline Sequence
Tag	EndlineSequence

Property	Value
Type	String
Default	\r\n
Required	yes

The end-of-line sequence. Supports \r \n \t escape codes.

Inherited AbstractProcessor options

Tag	Label	Default	Description
ExecuteProcessorConditionalExecution		true	When false (or expression evaluates false), the processor is skipped entirely. Supports PilotFish enhanced expressions.

4. Transaction attributes

None written by this processor.

5. Known quirks and caveats

readLine() strips all EOL – `BufferedReader.readLine()` removes \r, \n, and \r\n.

Trailing EOL – last line also gets the endline sequence appended (may add extra trailing newline).

Custom replace method – uses iterative indexOf/substring replacement instead of `String.replace`.

isConfigurationComplete – always returns true (no mandatory fields enforced).

6. Operational recommendations

Goal	Guidance
Cross-platform	Use \\r\\n for Windows, \\n for Unix/Linux targets
Binary data	Not suitable for binary streams; line-based reading corrupts binary content
EDI	Useful for normalizing EDI segment terminators

7. Source index

Topic	Path
End of Line Conversion	<code>com/pilotfish/eip/modules/file/EOLProcessor.java</code>
Module JAR	<code>xcs-modules-file-1.0.1.jar</code>

Deep-dive – End of Line Conversion Processor – 26R1.11 – August 2026.